

Sequence analysis

Pairwise local structural alignment of RNA sequences with sequence similarity less than 40%

Jakob Hull Havgaard¹, Rune B. Lyngsø², Gary D. Stormo³ and Jan Gorodkin^{1,*}

¹Center for Bioinformatics and Division of Genetics, IBHV, The Royal Veterinary and Agricultural University, Grønnegårdsvej 3, DK-1870 Frederiksberg C, Denmark, ²Department of Statistics, Oxford University, 1 South Parks Road, Oxford, OX1 3TG, UK and ³Department of Genetics, Washington University School of Medicine, Campus Box 8232, 4566 Scott Avenue, St Louis, MO 63110, USA

Received on September 2, 2004; revised on January 6, 2005; accepted on January 17, 2005

Advance Access publication January 18, 2005

ABSTRACT

Motivation: Searching for non-coding RNA (ncRNA) genes and structural RNA elements (eleRNA) are major challenges in gene finding today as these often are conserved in structure rather than in sequence. Even though the number of available methods is growing, it is still of interest to pairwise detect two genes with low sequence similarity, where the genes are part of a larger genomic region.

Results: Here we present such an approach for pairwise local alignment which is based on FOLDALIGN and the Sankoff algorithm for simultaneous structural alignment of multiple sequences. We include the ability to conduct mutual scans of two sequences of arbitrary length while searching for common local structural motifs of some maximum length. This drastically reduces the complexity of the algorithm. The scoring scheme includes structural parameters corresponding to those available for free energy as well as for substitution matrices similar to RIBOSUM. The new FOLDALIGN implementation is tested on a dataset where the ncRNAs and eleRNAs have sequence similarity <40% and where the ncRNAs and eleRNAs are energetically indistinguishable from the surrounding genomic sequence context. The method is tested in two ways: (1) its ability to find the common structure between the genes only and (2) its ability to locate ncRNAs and eleRNAs in a genomic context. In case (1), it makes sense to compare with methods like Dynalign, and the performances are very similar, but FOLDALIGN is substantially faster. The structure prediction performance for a family is typically around 0.7 using Matthews correlation coefficient. In case (2), the algorithm is successful at locating RNA families with an average sensitivity of 0.8 and a positive predictive value of 0.9 using a BLAST-like hit selection scheme.

Availability: The program is available online at <http://foldalign.kvl.dk/>
Contact: gorodkin@bioinf.kvl.dk

INTRODUCTION

Non-coding RNAs (ncRNAs) have just within the last few years been recognized as a highly abundant class of genes, involved

in various regulatory mechanisms in the cell (Eddy, 2002a). A variety of ncRNAs have received much attention recently, in particular the micro RNAs (Lagos Quintana *et al.*, 2001; Lau *et al.*, 2001; Lee and Ambros, 2001; Huttenhofer *et al.*, 2002). A related class of regulatory elements are the RNA structural *cis* regulatory elements (eleRNAs), and counting these together with the ncRNA genes, there exist about 380 such entries in the current Rfam database (Griffiths-Jones *et al.*, 2003). This class of genes is rapidly growing, and it is of increasing interest to detect them computationally.

Note that until just a few years ago ncRNAs were widely ignored in the gene prediction literature and to some extent in genome projects. In fact, even though ncRNAs now have attention in genome projects, little has been possible due to the high computational complexity of the methods for searching/screening for ncRNAs.

However, the recent introduction of the cm2hmm RNA family scanning tool is a breakthrough that seems to make it possible to systematically include ncRNA annotation in genome projects (Weinberg and Ruzzo, 2004a,b) at a higher level than just a few specific families (Lowe and Eddy, 1997, <http://genome.wustl.edu/eddy/tRNAscan-SE/>, 1999). These methods are based on stochastic context-free grammars (SCFGs) and hidden Markov models (HMMs). Previous approaches for scanning used in Rfam include the Infernal implementation by Eddy (2002b) that also allow for partial structural matches. Also, Brown (2000) constrained SCFGs with HMMs to match long RNA sequences (small subunit rRNA). A faster method, RSEARCH, is also useful (Klein and Eddy, 2003). Whereas these methods are well suited to locate known ncRNA families, they are not likely to find novel classes of ncRNAs. An approach suited for finding novel ncRNAs is QRNA which exploits a semi-conserved pairwise sequence alignment to detect significant covariance between the sequences (Rivas and Eddy, 2001). A recent approach using folding energy in related sequences has also been introduced to scan for ncRNAs (Washietl and Hofacker, 2004).

Other methods can also find a common motif in a set of multiple related Sequences. (Macke *et al.*, 2001; Eddy, 2002b; Perriquet

*To whom correspondence should be addressed.

et al., 2003; Pavesi et al., 2004; Ji et al., 2004). However, these are not expected to work well on just two sequences. Other methods work well in finding the secondary structure once a good multiple alignment has been obtained (Knudsen and Hein, 1999; Hofacker et al., 2002).

If the structure searched for is energetically different from its sequence context, energy based approaches such as that of Hofacker et al. (2004b) might be of interest. Nonetheless, all the methods mentioned above require some sort of biological knowledge that goes into the problem prior to prediction. Here we address the basic problem of finding common local structure between just two sequences. To test the method we try to remove prior knowledge by considering sequences with low similarity, <40%. Furthermore, as some ncRNA sequences have a folding energy very different from their surrounding sequence context, we also discard such sequences. Sequences with low sequence similarity was also the major challenge for Weinberg and Ruzzo (2004b).

In principle the algorithm by Sankoff (1985) copes with many of the limitations mentioned above, but it is computationally intractable for more than two sequences. An early implementation of the Sankoff algorithm, FOLDALIGN, was in principle designed to handle the problem, but, was due to the computational capacities at that time, limited to stem-loop structures only (Gorodkin et al., 1997a,b). Later heuristics were employed to speed up the computation (Gorodkin et al., 2001a,c). Other methods have implemented pairwise versions of the Sankoff algorithm (Mathews and Turner, 2002; Holmes, 2004; Hofacker et al., 2004a). Here, we not only include (non-pseudoknotted) bifurcated structures, but we also present an effective approach for conducting mutual simultaneous scan of two sequences to search for common local structural motifs.

Furthermore, in contrast to the former version of FOLDALIGN, which is limited to maximizing a score, the version presented here employs substitution matrix schemes similar to Klein and Eddy (2003) including parameters corresponding to energy terms with similarities to those in Dynalign (Mathews and Turner, 2002). However, FOLDALIGN still maximizes a score which enables it to perform local alignments.

ALGORITHM

The FOLDALIGN is a simplified version of the Sankoff algorithm (Sankoff, 1985) which uses dynamic programming to locally align two sequences of lengths L_1 and L_2 . Previous versions (Gorodkin et al., 1997a,b, 2001b,c) used base-pair substitutions and a simple stacking scoring scheme. It was limited to stem-loop structures due to the computational complexity. The version presented here can find any non-pseudoknotted structure. Furthermore, the scoring scheme has been extended with some of the rules and parameters used in energy minimization (Mathews and Turner, 2002; Mathews et al., 1999; Xia et al., 1998; Zuker and Stiegler, 1981) as well as the computation of substitution matrices which now has similarities to RIBOSUM (Klein and Eddy, 2003) and BLOSSUM (Henikoff and Henikoff, 1992).

Computing scores and costs

The full recursion for computing the score $D_{ij,kl}$ of aligning subsequence (i, j) to subsequence (k, l) can be

written as

$$D_{ij,kl} = \max \left\{ \begin{array}{l} D_{(i+1)(j-1),(k+1)(l-1)} + S_{n_i n_j, n_k n_l}, \\ D_{i(j-1), (k+1)(l-1)} + S_{-n_j, n_k n_l}, \\ D_{(i+1)j, (k+1)(l-1)} + S_{n_i -, n_k n_l}, \\ D_{(i+1)(j-1), k(l-1)} + S_{n_i n_j, -n_l}, \\ D_{(i+1)(j-1), (k+1)l} + S_{n_i n_j, n_k -}, \\ D_{(i+1)(j-1), kl} + S_{n_i n_j, -}, \\ D_{ij, (k+1)(l-1)} + S_{- -, n_k n_l}, \\ D_{(i+1)j, (k+1)l} + S_{n_i -, n_k -}, \\ D_{i(j-1), k(l-1)} + S_{-n_j, -n_l}, \\ D_{(i+1)j, k(l-1)} + S_{n_i -, -n_l}, \\ D_{i(j-1), (k+1)l} + S_{-n_j, n_k -}, \\ D_{(i+1)j, kl} + S_{n_i -, -}, \\ D_{i(j-1), kl} + S_{-n_j, -}, \\ D_{ij, (k+1)l} + S_{- -, n_k -}, \\ D_{ij, k(l-1)} + S_{- -, -n_l}, \\ \max_{\substack{i < n < j-1 \\ k < m < l-1}} \{ D_{in, km} + D_{(n+1)j, (m+1)l} \} \end{array} \right. \quad (1)$$

where n_i is the nucleotide at position i . $S_{n_i n_j, n_k n_l}$ is the cost of substituting (n_i, n_j) with (n_k, n_l) and while simultaneously folding the two subsequences. The substitution cost, $S_{n_i n_j, n_k n_l}$ is in general computed dynamically depending on the structural context. The computation makes use of static parameters such as base-pair substitution costs and energy parameters. In contrast to the previous implementation, the following cases are now used: (a), (c), (d), (f) and (g), where (c) is used for a single base-pair insertion and (g) gives bifurcation. Limits have been placed on in which contexts (g) is calculated.

The dynamical computation of $S_{n_i n_j, n_k n_l}$ depends on the five structural contexts: hairpin-loop, stem, internal-loop, bulge, and multibranch-loops (Fig. 1). In some contexts a previous computed cost S needs to be recalculated. How the cost $S_{n_i n_j, n_k n_l}$ is computed from the score matrices $S(n_i, n_j, n_k, n_l)$ is outlined below. The context of each $D_{ij,kl}$ is stored in a separate matrix. The energy rules used in the cost calculation are a subset of the Turner energy rules described in the mfold documentation (Zuker et al., 1999; Mathews et al., 1999).

Hairpin-loop. An alignment is always initiated in the hairpin-loop context. The hairpin-loop context is the single-stranded part of a structure which is closed by two nucleotides that base-pair. The cost for aligning two hairpin-loops is:

$$S_{hp} = S_{\text{substitution}} + S_{\text{length}} + S_{\text{stack}}, \quad (2)$$

where the cost $S_{\text{substitution}} = \sum S_{ss}(n_i, n_k)$ is calculated by adding up the substitution and gap cost for each pair of nucleotides or gap in the loop. The single-strand substitution costs $S_{ss}(n_i, n_k)$ are stored in the single-strand substitution matrix in the score matrix.

The cost $S_{\text{length}} = S_{hp_length}(j - i + 1) + S_{hp_length}(l - k + 1)$ is a loop size dependent cost, calculated from energy parameters stored in the hairpin column of the loop length costs table in the score matrix.

The default minimum hairpin-loop length is three nucleotides. For hairpin-loops longer than three nucleotides there is an energy cost for stacking the two unpaired nucleotides (n_i, n_j) at the end of the loop on top of the base-pair (n_{i-1}, n_{j+1}) which closes

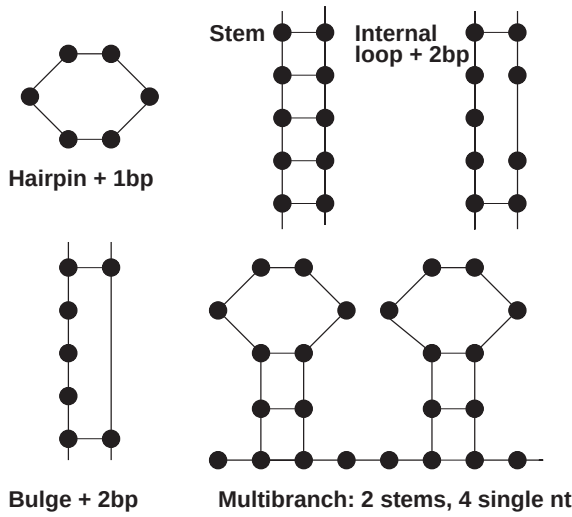


Fig. 1. The cost depends on five structural contexts: hairpin-loop, stem, internal-loop, bulge and multibranch-loop.

the hairpin-loop. The cost $S_{\text{stack}} = S_{\text{hp_stack}}(n_i, n_j, n_{i-1}, n_{j+1}) + S_{\text{hp_stack}}(n_k, n_l, n_{k-1}, n_{l+1})$ is calculated from the energy parameters stored in the hairpin close matrix in the score matrix. The hairpin-loops of both sequences have to be longer than three nucleotides before the stacking cost is added. The stacking values have a non-GC cost added. This cost can be explained by the distribution of the hydrogen bonds in the nearest neighbor model (Xia *et al.*, 1998).

Stem. A stem is a series of stacked base-pairs at least two base-pairs long. A single base-pair is not allowed and is recalculated as part of the surrounding loop. The score for aligning two stems has two parts:

$$S_{\text{bp}} = S_{\text{substitution}} + S_{\text{stack}}. \quad (3)$$

The cost $S_{\text{substitution}} = \sum S_{\text{bp}}(n_i, n_j, n_k, n_l)$ is the sum of the costs for substituting the base-pairs in one subsequence with the base-pairs in the other. These base-pair substitution costs, S_{bp} , are stored in the base-pair substitution matrix in the score matrix.

The stacking cost of the base-pair (i, j) depends on the base-pair $(i-1, j+1)$. For the last base-pair special rules apply (see internal-loops and multibranch-loops). The cost $S_{\text{stack}} = \sum S_{\text{stacking}}(n_i, n_j, n_{i-1}, n_{j+1}) + \sum S_{\text{stacking}}(n_k, n_l, n_{k-1}, n_{l+1})$ is the sum of the two independent sums of stackings in the two stems. The S_{stacking} cost is found in the stacking matrix in the score matrix.

When aligning two stems to each other, two gaps can be inserted into one of the subsequences making that stem one base-pairs longer than the other. This is only allowed when the stem is at least two base-pairs long and the inserted base-pair is surrounded by standard base-pairs (AU, GC, GU). An example of an insert base-pair calculation is

$$\begin{aligned} S_{\text{bp_insert}} = & S_{\text{bp}}(n_i, n_j, n_k, n_l) + S_{\text{ss}}(n_{i+1}, -) + S_{\text{ss}}(n_{j-1}, -) \\ & + S_{\text{bp}}(n_{i+2}, n_{j-2}, n_{k+1}, n_{l+1}) \\ & + S_{\text{stack}}(n_{i+2}, n_{j-2}, n_{i+1}, n_{j-1}) \\ & + S_{\text{stack}}(n_{i+1}, n_{j-1}, n_i, n_j) + S_{\text{stack}}(n_{k+1}, n_{l-1}, n_k, n_l). \end{aligned} \quad (4)$$

Internal-loop. Internal-loops and bulges are single-stranded parts of an RNA structure surrounded by stems. A bulge is a

single-strand region in only one side of the structure (Fig. 1). An internal-loop contains single-stranded nucleotides on both sides of the structure. In the alignment an internal-loop aligned to a bulge is treated as two internal-loops. The score of an internal-loop has four parts:

$$S_{\text{il}} = S_{\text{substitution}} + S_{\text{length}} + S_{\text{asymmetry}} + S_{\text{stack}}. \quad (5)$$

The cost $S_{\text{substitution}} = \sum S_{\text{ss}}(n_i, n_k) + \sum S_{\text{ss}}(n_j, n_l)$ is the sum of the substitution costs for each pair of nucleotides in the loop. The substitution costs are those from the single-strand substitution matrix.

The length cost is a function of the sum of lengths of each side of the internal-loops. For an internal-loop spanning positions $i' - i$ and positions $j' - j$, the internal-loop lengths are $\Delta l_i = i - i' + 1$ and $\Delta l_j = j - j' + 1$. The cost $S_{\text{length}} = S_{\text{internal}}(\Delta l_i + \Delta l_j) + S_{\text{internal}}(\Delta l_k + \Delta l_l)$ is calculated using the energy parameters stored in the internal column of the loop length costs table.

The two sides of an internal-loop do not have to have same lengths, but the length difference has a cost which is proportional to the length difference. $S_{\text{asymmetry}} = S_{\text{asym}}(|\Delta l_i - \Delta l_j|) + S_{\text{asym}}(|\Delta l_k - \Delta l_l|)$ is the sum of the two asymmetry costs.

The first pair of nucleotides in an internal-loop stack with the last base-pair in the previous stem. Likewise, the last pair of nucleotides in the loop stack on the first base-pair in the next stem. The cost $S_{\text{stack}} = S_{\text{il_stack}}(n_{i'}, n_{j'}, n_{i'+1}, n_{j'-1}) + S_{\text{il_stack}}(n_{k'}, n_{l'}, n_{k'+1}, n_{l'-1}) + S_{\text{il_stack}}(n_i, n_j, n_{i-1}, n_{j+1}) + S_{\text{il_stack}}(n_k, n_l, n_{k-1}, n_{l+1})$ is the sum of these four costs for the two structures in the alignment. The internal-loop stacking values are those in the internal-loop stacking matrix in the score matrix. The internal-loop stacking parameters have also had a non-GC cost added.

Bulge-loop. Bulges are similar to internal-loops in many ways. The cost of a bulge is therefore calculated as an internal-loop until it is certain that it really is a bulge. Then the bulge cost is recalculated. The cost of a bulge is:

$$S_{\text{bl}} = S_{\text{substitution}} + S_{\text{length}} + S_{\text{stack}}. \quad (6)$$

The substitution cost is the same for the bulges and the internal-loops. The cost S_{length} depends on the two bulge lengths and is stored in the bulge column of the loop length cost table. Each subsequence length is calculated separately, and their costs are added together. If the bulges in both sequences are one-nucleotide long or if there is a one-nucleotide bulge in one sequence and no bulge in the other, then the surrounding base-pairs are allowed to stack as if the bulges were not there. If the bulge length is >1 nucleotide then there is an extra cost for surrounding base-pairs which are not G-C base-pairs.

Multibranch-loop. Multibranch-loops are regions where ≥ 2 stems meet. The cost of a multibranch-loop is:

$$\begin{aligned} S_{\text{mbl}} = & S_{\text{substitution}} + S_{\text{mbl_closing}} + (n_{\text{stems}} - 2)S_{\text{stem}} \\ & + n_{\text{singlenucleotides}} \times S_{\text{nucleotide}} + S_{\text{stack}}, \end{aligned} \quad (7)$$

where $S_{\text{substitution}}$ is calculated using the single-strand substitution matrix. The cost $S_{\text{mbl_closing}}$ is added when a multibranch-loop is closed by a base-pair, S_{stem} is for each additional pair of conserved

stems after the first two, and $S_{\text{nucleotide}}$ is added to each single-stranded nucleotide in the loop. Single-stranded nucleotides next to a stem are stacked. The cost is taken from the dangling end matrices. If a nucleotide can stack on two stems, the most favorable stacking is chosen. There is an extra cost for all stems not ending with a G–C base-pair. The stackings are summed for both subsequences into the S_{stack} cost.

Backtracking. When the score D has been calculated for all values of (i, j, k, l) , the structure of the best scoring alignment can be found by backtracking the D matrix.

Score matrices and parameters

Two types of parameters are used: energy and sequence similarity. The energy parameters are taken from mfold (Zuker et al., 1999; Mathews et al., 1999). All energy parameters have been multiplied by -10 mol/kcal and rounded to the nearest integer. The similarity parameters are log-odds substitution rates similar to RIBOSUM-matrices (Klein and Eddy, 2003) with two exceptions. No intra-cluster counts are allowed, and no low similarity cut-off is used (Henikoff and Henikoff, 1992). The two substitution matrices are scaled independently. The alignment used for making the substitution matrices is the 1995 version of the small subunit ribosomal RNA alignment from the European Ribosomal RNA Database (Van de Peer et al., 1994) supplied and described by Klein and Eddy (2003). The problem of estimating parameters from one family and applying these to others has previously been addressed, where it was found that parameters estimated from rRNA and tRNA could be applied to HIV-1 RNA analysis (Knudsen et al., 2004).

The stacking bonus can be estimated from a matrix just like the substitution costs. In this way the bonus becomes the cost of substituting one stacked base-pair with another stacked base-pair. The number of different stacked base-pairs is $6^4 = 1296$ which is a very high number of parameters to estimate. Assuming that the stackings are independent of each other, the substitution cost is approximated as the substitution cost plus the two log-odds stackings bonus. This gives only $6^2 = 36$ parameters to estimate. When testing the program, we had to choose between the experimentally measured stacking parameters and those estimated from the SSU alignment. We chose to use the energy stacking parameters.

Each score matrix has five parameters which are not determined experimentally or estimated from multiple alignments: gap opening, gap elongation, similarity clustering percentage, single-strand substitution matrix scale and base-pair substitution scale. These five values need to be chosen before running FOLDALIGN. If one wishes to conduct the structural alignment unbiased by sequence similarity, the single-strand substitution matrix and the base-pair substitution matrix are simply scaled to zero (Hofacker et al., 2004a).

Mutual scan of two sequences and the algorithmic complexity

The algorithm as presented so far has a time complexity of $L_1^3 L_2^3$ and a memory complexity of $L_1^2 L_2^2$. For L_1 and L_2 longer than a few hundred nucleotides both time and memory become prohibitive. Therefore we use the same two heuristics as in the previous versions: the maximum length of the RNA-motif is limited to λ nucleotides, and the maximum length difference between two subsequences being compared is limited to δ nucleotides. This lowers

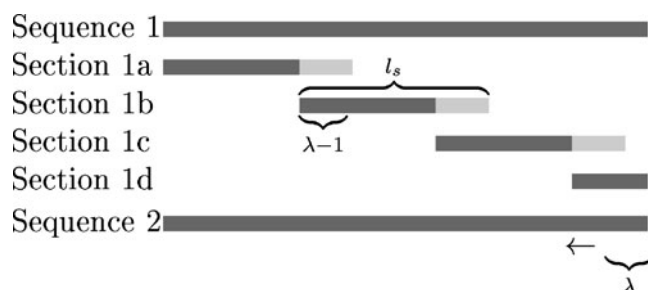


Fig. 2. The first sequence is split into smaller sections — a, b, c and d. Each section overlaps the next section with $\lambda - 1$ nucleotides. Each section is aligned to the second sequence. During the alignment a window of λ nucleotides from sequence 2 is aligned to the current section. Then the window is moved by one-nucleotide, and the new window is aligned to the section. By starting from the end and moving toward the beginning of sequence 2 the information stored about the previous position can be easily reused. When all of sequence 2 has been aligned to the section, the alignment process starts over with the next section. Due to the sequence overlap FOLDALIGN with the default section length, $l_s = 2\lambda - 1$, runs half as fast as when the section length equals the length of sequence 1.

the time complexity to $L_1 L_2 \lambda^2 \delta^2$. The memory complexity becomes $L_1 L_2 \lambda \delta$.

The memory complexity of $L_1 L_2 \lambda \delta$ can be reduced further by splitting the first sequence into smaller subsections. Each section is then aligned to a sliding window on the second sequence (Fig. 2). Since the program aligns all subsequences of lengths up to λ , each section has to overlap the next section by $\lambda - 1$ nucleotides. The alignments at each position k in sequence 2 are only dependent on the positions (k) to $(k + \lambda - 1)$. It is therefore only necessary to keep a window of size λ of the D matrix in the memory at any time. To ease the implementation the second sequence is scanned from end to start. In this way the values calculated for the $(k + 1)$ to $(k + \lambda)$ window can be reused directly. The full two sequences are still kept in memory giving the program a memory requirement of $O(l_s \lambda^2 \delta + L_1 + L_2)$, where l_s is length of the subsections.

Splitting one sequence into overlapping sections increases the run time of the program. If the sequence is split into $\lceil (L_1 - (\lambda - 1)) / (l_s - (\lambda - 1)) \rceil$ sections of length l_s , the time complexity becomes $O((L_1 l_s / l_s - \lambda) L_2 \lambda^2 \delta^2)$. The FOLDALIGN uses $l_s = 2\lambda$ as the default l_s value. This subsection length gives a doubling of the run time compared to not splitting the sequence. When the alignment step is done, the regions of interest must be realigned one by one to get the structure. This adds another $O(\lambda^4 \delta^2)$ per hit region. Currently, only the best scoring hit is backtracked.

The total run time scales as $O(L_1 L_2 \lambda^2 \delta^2 + N \lambda^4 \delta^2)$, where N is the number of structures to be backtracked.

To speed up the program, parts (b) and (e) of Equation (1) have not been implemented. This does not affect the time and memory complexities. Since the alignments of cases (b) and (e) can be done by combinations of some of the other cases this does not change the types of structures that can be aligned.

A constraint has also been placed on when part (g) of Equation (1) is calculated. The (g) part combines two substructures. To speed up the the program the calculation is only performed when the left

part of the structure ($D_{in,km}$) has base-pairs between (i & n) and (k & m). The right part of the structure ($D_{(n+1)j,(m+1)l}$) is limited to the cases where j is base-paired and l is base-paired. These constraints do not limit the kinds of structure that can be found since any unconstrained structures that can be found, can also be found as a constrained structure extended using the (a), (c), (d) and (f) parts of Equation (1).

Selection of a hit region

It is possible for two sequences to share more than one common locally conserved structure. Two such structures cannot overlap each other in both sequences. A list of the best scoring, non-overlapping regions in the two sequences can be made by the following simple algorithm. In a matrix of all the best local alignment scores:

- (1) Find the best scoring alignment $D_{ij,kl}$.
- (2) Remove all alignments $D_{i'j',k'l'}$ for which
 $i \leq i' \leq j$ or $i \leq j' \leq j$,
and
 $k \leq k' \leq l$ or $k \leq l' \leq l$.
- (3) Repeat until all alignments have been removed or a predetermined threshold is reached.

When the list of alignments has been made, a cutoff for considering an alignment real or random has to be chosen. For one scan this can sometimes be seen simply by looking at the score, but when there are more than a few scans an automated method is needed. Three approaches are tested:

The best n alignments: For each scan the n best scoring alignments are chosen. Simply selecting the best alignment from each scan is the $n = 1$ case. This is the default behavior of the FOLDALIGN program.

Z-value: The Z-score for all alignments in the list is calculated based on all the scores in the (i, k) matrix.

Extreme value distribution: The scores from the alignment are assumed to follow an extreme value distribution in the same way as the scores from the BLAST program (Karlin and Altschul, 1990). The extreme value distribution is characterized by two parameters Λ and K (Λ is usually called λ in other texts). The island method allows Λ and K to be estimated from a few random alignments scores (Olsen *et al.*, 1999; Altschul *et al.*, 2001). Due to the high time complexity of the algorithm Λ and K are sometimes estimated directly from the scores of the alignments in the alignment list. The calculations follow those in Altschul *et al.* (2001). The mean $\hat{D}$, of the alignment scores for alignments with a score above a threshold C is calculated. Λ and K are then estimated as:

$$\Lambda_C = \ln \left(1 + \frac{1}{\hat{D} - C} \right), \quad K = \frac{n_{D>C} \exp^{\Lambda_C C}}{L_1 L_2},$$

where $n_{D>C}$ is the number of alignments in the list scoring better than the threshold C . The estimated Λ and K are now used to calculate a rough estimate of the probability for seeing a random alignment with the score D or better.

$$\text{Prob}(\text{Score} \geq D) \approx 1 - \exp(-K L_1 L_2 \exp(-\Lambda D)). \quad (8)$$

The probability score should not be taken as a good estimate of the real probability.

The work in Heyer (2000) also describes related statistical properties of FOLDALIGN scores.

Performance evaluation

Two cases of performance are evaluated: the structure of the alignment and the localization of alignments. The structure performance measures how well the predicted structure fits to a known curated structure. The localization performance tells us how well the correct alignments are separated from the random alignments. This is our ability to select the right region among many hits. Three measures are used to evaluate the predictions: the Matthews correlation coefficient (CC) (Matthews, 1975), the sensitivity (SEN) and the positive predictive value (PPV) (sometimes called the specificity). These are defined as:

$$\begin{aligned} \text{CC} &= \frac{P_t N_t - P_f N_f}{(P_t + P_f)(P_t + N_f)(N_t + P_f)(N_t + N_f)}, \\ \text{SEN} &= \frac{P_t}{P_t + N_f}, \quad \text{PPV} = \frac{P_t}{P_t + P_f}, \end{aligned} \quad (9)$$

where P_t , P_f , N_f and N_t are the numbers of true positives, false positives, false negatives and true negatives. These are defined for each case below. For the structure predictions the correlation coefficient can be approximated as the geometric mean of the sensitivity and the positive predictive value, $\text{CC} = \sqrt{\text{SEN} \times \text{PPV}}$ (Gorodkin *et al.*, 2001c).

Structure. The evaluation of structure predictions follow Mathews and Turner (2002) and Mathews *et al.* (1999). A predicted base-pair between positions i and j is considered as correctly predicted, true positive, if a base-pair is annotated between i and j , $i \pm 1$ and j , or i and $j \pm 1$. Predicted base-pairs which pair other positions are counted as false positives. Annotated base-pairs which are not predicted are counted as false negatives.

Localization. A predicted hit (alignment) is counted as a true positive, if for both sequences at least half the positions predicted as part of the alignment overlap an annotated motif. An alignment where less than half the predicted positions are annotated on one or both sequences is counted as a false positive alignment. From the annotation of the sequences it is known which regions should be aligned. Any alignments between two RNAs from the same family missing are counted as false negative alignments. For correct alignments and completely missed alignments the family of the alignment is taken from the annotations. A false positive alignment which overlaps an annotated gene with at least half of its positions is counted as belonging to the family of the annotated RNA. All other false positives are counted as a part of the unknown family. Predicted alignments between two different types of RNAs are counted as unknown false positives.

IMPLEMENTATION

The FOLDALIGN algorithm is implemented as a C++ program. The program has been tested on machines running different types of Linux. The implementation uses an extra matrix besides the score and the context matrices already described. This matrix, the length matrix, keeps track of the number of times the same context has been used in a row. This is used to speed up the backtracking during

alignment. The memory requirement of the length matrix scales in the same way as the other two matrices.

The program has a number of command-line options. The most important ones control the λ and the δ parameters. The number of overlapping nucleotides during scans can also be set. These three parameters greatly influence the memory consumption and the run time of the program. By default λ and δ are set to the sequence length. The number of overlapping nucleotides l_s is set to 2λ by default. If this is longer than the sequence length, then the sequence length is used. The program performs a local alignment by default, but a global alignment can be chosen using an option. A score matrix different from the default can be read from a file. The default score matrix file is part of the software package. Using option 'help' will print a small help text.

The default sequence input format is fasta. The program can also read a space/tab separated file with the format `name<space>sequence<space>not_used<space>comments`. A third file format is the pair format. In this format each line in the input file contains `name1<space>sequence1<space>name2<space>sequence2<space>options`. The program will align sequence 1 with sequence 2 using the options specified. Finally two sequences can be specified on the command-line.

RESULTS

The FOLDALIGN predicts a local alignment and structure for two sequences containing a common conserved RNA-motif. The version presented here has been improved over earlier versions (Gorodkin *et al.*, 1997b) in two ways.

The memory complexity of the algorithm has been lowered to a point where sequences of any length can be scanned given enough time. However, the length of the RNA-motifs is still limited by the memory of the computer. As in the previous versions of the algorithm two heuristics are used to limit the calculation time and the memory consumption. The RNA-motif length is limited to λ nucleotides. This speeds up the calculation, but more importantly it allows the sequences to be aligned in smaller sections (Fig. 2). The second heuristic limits the length difference between two subsections being aligned. Limiting this length difference to δ nucleotides greatly lowers the computation time and also lowers the memory requirement. To illustrate the necessity and efficiency of these heuristics, two sequences, 300 nucleotide long, were aligned using the different time and memory complexities shown in Table 1. The machine used was an SGI Altrix 3000 with 64 Intel Itanium 2900 MHz processors and 190 GB of memory. Even with the relatively short sequences used in this example there is a large drop in the needed time and memory. The memory requirement is reduced to <1% of the unlimited case. For the longer sequences used later the savings are even greater. This represents an effective constraint on the Sankoff algorithm when the motif is limited in size.

The second improvement is the use of a subset of the rules used in energy minimization folding. Instead of just using single-strand substitutions, base-pair substitutions and a limited form of stacking, the algorithm now also utilizes multiple forms of stacking and length dependencies of different structural elements. The energy parameters used are a subset of those used by mfold (Zuker *et al.*, 1999; Mathews *et al.*, 1999). The substitution matrices are based on the RIBOSUM-matrices (Klein and Eddy, 2003), but follow Henikoff and Henikoff (1992) more strictly.

Table 1. The time and memory needed by the FOLDALIGN program

Case	Time	Memory	Time	Memory
No heuristics	$L_1^3 L_2^3$	$L_1^2 L_2^2$	~3 days	27.8 GB
Scanning	$L_1 L_2 \lambda^2 \delta^2$	$\lambda^3 \delta$	49 min 31 s	273 MB

Two sequences, 300 nucleotide long, were aligned. In the scanning run $\lambda = 100$ and $\delta = 15$ were used.

The program was tested using sequences selected using two main criteria. Two RNA-motifs being compared must be <40% similar. Furthermore, the folding energy of the RNA-motifs must not be significantly different from the folding energy of the surrounding sequence. The Z-value for the folding energy of the motif compared to the folding energy of the surrounding sequence has to be -2 or more. To keep the computation time down a motif-length limit of ~150 nucleotides was used. Sequences were extracted from the 5S rRNA database (Szymanski *et al.*, 2002), the tRNA database (Sprinzl *et al.*, 1998) and the U1 database (Zwieb, 1996). To get the surrounding sequence an exact match for each sequence in the databases was found in GenBank (Benson *et al.*, 2004). For sequences with multiple hits an entry was chosen at random. Furthermore, sequences from the purine and THI families of RFAM version 5.0 (Mandal *et al.*, 2003; Mandal and Breaker, 2004; Rodionov *et al.*, 2002; Miranda Rios *et al.*, 2001; Sudarsan *et al.*, 2003; Winkler *et al.*, 2002; Kubodera *et al.*, 2003; Griffiths-Jones *et al.*, 2003) were used. The RFAM structures were compared and corrected to fit those in the publications. Unpaired nucleotides at the ends of the RFAM structures were removed. The surrounding sequence of the purine and THI sequences were located using their RFAM names. RNA-motifs which have a sequence similarity of $\geq 90\%$ are considered to be the same motif. Each RNA-motif was allowed to be in a maximum of three sequence pairs. Due to the high number of known tRNA sequences these were only allowed in one pair. Two versions of the dataset were made: One consisting of only the RNA-motifs, and one containing 500 nucleotide long sequences each containing one of the RNA-motifs. Some of the long sequences contain RNAs other than those in the dataset. These were annotated, using GenBank and RFAM, in the long sequences but not included in the motif dataset. In some cases a sequence contains one or two other RNAs which were only partially included. These RNAs were not annotated. Hits to such partial genes are counted as false positives. All GenBank sequences were manually checked. The motif dataset contains 99 RNA pairs and the long dataset also has 99 sequence pairs but a total of 295 gene pairs. As an extra test set eight sequence pairs were taken from the SRP database (Rosenblad *et al.*, 2003, <http://psyche.edu/dbs/SRPDB/SRPDB.html>). Since the SRP motifs are longer than the other motifs used, 1000 nt long sequences were extracted from GenBank.

The algorithm's performance was measured for two different cases: structure and selection. To align two subsequences correctly the algorithm has to find a common structure. How well the predicted structure fits with the structure annotated in the database is the first performance test. The second performance test tells how well the algorithm locates RNA-elements in long sequences.

Structure predictions

To evaluate the algorithm's ability to find the correct structure shared between two sequences the RNA-motif sequences were aligned

Table 2. Structure prediction performance for FOLDALIGN and Dynalign

RNA type	Number of pairs	FOLDALIGN			Dynalign			FOLDALIGN family			Dynalign family		
		CC	SEN	PPV	CC	SEN	PPV	CC	SEN	PPV	CC	SEN	PPV
All	99	0.72	0.68	0.78	0.69	0.69	0.71						
5S rRNA	2	0.70	0.66	0.75	0.61	0.59	0.65	0.74	0.61	0.95	0.64	0.46	0.93
Purine	5	0.93	0.91	0.95	0.90	0.87	0.95	0.93	0.92	0.94	0.97	0.99	0.96
THI	21	0.45	0.31	0.69	0.45	0.40	0.51	0.61	0.58	0.65	0.56	0.52	0.60
U1	6	0.65	0.67	0.63	0.69	0.77	0.63	0.71	0.77	0.66	0.71	0.77	0.65
tRNA	65	0.86	0.84	0.89	0.82	0.83	0.82	0.86	0.84	0.89	0.84	0.84	0.83

The main columns are the RNA type and the overall average, All. The FOLDALIGN and Dynalign columns are the performances when one set of parameters is used to predict the structures of all families. The FOLDALIGN family and Dynalign family show the performance when an optimal set of parameters is used for each family. For each main column there are three subcolumns with the correlation coefficient CC, the sensitivity SEN and the positive predictive value PPV.

without surrounding sequence. λ was set to the sequence length, and δ to 25. Different values of the five parameters were tested. The performance evaluation for the score matrix which best predicts the structures can be seen in Table 2 in the column FOLDALIGN. The best results for predicting each RNA-family separately can be seen in the FOLDALIGN family column. The correlation coefficient, CC for predicting the structures of 5S rRNA and U1 families are close to 0.7 for the same set of parameters. The tRNA performs a little better. The performance on the Purine family is better than on the other four families. The sequences in this set all have a very well conserved structure. There are no long inserts or deletions, and no extra or missing stems which explains the better performance. The THI family performs worse than average. The structure of the THI element is less well conserved. It can have very large inserts in stem 2, and stem 3 is often missing (see Figure 2 in Rodionov *et al.*, 2002). The most important parameter for the structure prediction is the gap opening cost, and the weight of the single-strand substitution matrix. The gap elongation cost is of less importance and was usually fixed to half the gap opening cost. The single-strand substitution matrix affects the performance more than the base-pair substitution matrix. The best performing score matrix without any similarity has a CC of 0.6. This shows that even though sequence similarity between the sequences is low there is still some useful information left, which may explain why FOLDALIGN does slightly better than Dynalign which uses no sequence information.

To compare how well FOLDALIGN performs compared to Dynalign the motif dataset was aligned using Dynalign. Dynalign has two variable structural parameters: the gap penalty and whether to allow single base-pair insertions or not. A number of test alignments showed that allowing base-pair insert gave the best performance. Single base-pair insertions were allowed for alignments used. The sequence pairs were aligned using gap penalties between 0 and 2.0 kcal/mol with 0.1 kcal/mol increments. The performance of Dynalign using the best set of parameters to predict the structures of all families can be seen in the Dynalign column of Table 2. The best performances for each family can be seen in the Dynalign family column. While there are differences for the individual families, the overall performances of the two programs are very similar.

Dynalign has a constraint parameter, M . The positions of aligned nucleotides must be within M nucleotides of each other. It is similar to the FOLDALIGN δ constraint in the sense that two subsequences

aligned can only have a length difference of M nucleotides. Therefore it could be argued that comparing a FOLDALIGN alignment made with $\delta = 25$ to a Dynalign alignment made with $M = 15$ is not fair. The FOLDALIGN was rerun with the same parameters as in the FOLDALIGN column except that δ was changed to 15. There was no difference in performance.

Table 3 shows the time and memory needed by FOLDALIGN and Dynalign to align three sequence pairs of different lengths. It can be seen that while FOLDALIGN is much faster, it also uses more memory. Dynalign owes its memory efficiency to the M parameter. By requiring that the sequences are aligned to within M nucleotides instead of just limiting the length differences, one of the dimensions of the dynamic programming matrices is reduced from alignment length to M . Since FOLDALIGN does not have the prealignment requirement, the whole sequence section has to be kept in memory resulting in the higher memory requirement.

The parameters found to be optimal for predicting structures were used to predict the structures of the sequences in the SRP dataset. The average CC for the structure prediction was 0.65. The sensitivity and positive predictive values were 0.58 and 0.74, respectively.

Localization

To test FOLDALIGN's performance as a local alignment tool the dataset containing the RNA-motifs with their surrounding sequence was aligned. A number of different parameters were tested. λ was chosen to be approximately the size of the RNA-motif, and δ was fixed at 15. The most important parameter was the gap opening. It was found that higher gap opening penalties were needed for localization compared to structure prediction. Similarity scores was found to improve the localization performance.

To get a better estimate of the probabilities that the alignments found are random alignments, the sequences were shuffled ten times in a way which conserved the dinucleotide distribution (Workman and Krogh, 1999).

Table 4 shows the performance for hit selection using Z-value scores and probabilities based on extreme value distributions. For each sequence pair the parameters of the two distributions were estimated from the alignment of the shuffled sequences. Only hits with a Z-value >8 were reported. The performance is perfect except for tRNAs and THI elements.

The results reported in the Z-value column assume that the Z-values from one scan can be compared with those from another.

Table 3. The time and memory needed for FOLDALIGN and Dynalign to align two sequences

RNA type	Gb accession 1	Gb accession 2	Length 1	Length 2	FOLDALIGN		$\delta = 25$		Dynalign	
					$\delta = 15$					
					CPU time	Memory (MB)	CPU time	Memory (MB)	CPU time	Memory (MB)
tRNA	AF047724	AD000019	72	72	16 s	45	28 s	64	6 min 41 s	18
THI	AE007806	AP005276	91	93	46 s	98	1 min 30 s	138	19 min 42 s	25
U1	AC004546	X63783	164	163	7 min 37 s	521	16 min 18 s	779	2 h 10 min 1 s	70

The machine used has two 2.4 MHz Intel XEON processors and 2 GB of memory.

Table 4. The location performance for each family

RNA	Z-value					Extreme value distribution				
	P_t	P_f	N_f	SEN	PPV	P_t	P_f	N_f	SEN	PPV
5S rRNA	2	0	0	1.00	1.00	2	0	0	1.00	1.00
Purine	5	0	0	1.00	1.00	5	1	0	1.00	0.83
THI	13	6	8	0.62	0.68	8	1	13	0.38	0.89
U1	6	0	0	1.00	1.00	6	1	0	1.00	0.86
tRNA	134	23	109	0.55	0.85	174	44	69	0.72	0.80
Unknown		30					25			

This shows how well the correct alignments are selected from the random alignments. Two methods were used to predict the alignments that were hit. The first used Z-scores and the second a probability estimated from the extreme value distribution. The true positives, P_t , are alignments where at least half of the two RNAs are predicted to align. The false positives, F_p , are the hits where one RNA of this type either overlaps less than half of another RNA or something that has not been annotated. When the two regions, which are not annotated, are aligned then they are counted as being part of the 'Unknown' family. False negatives, F_n , is the number of alignments which should have been found but were not. SEN is the sensitivity and PPV the positive prediction value.

This is most likely not the case. The FOLDALIGN scores are expected to follow an extreme value distribution like the scores from BLAST (Karlin and Altschul, 1990; Heyer, 2000). The probabilities of getting random alignments with the score of the predicted alignments are estimated. The estimates are very crude, and the estimated probabilities are far from correct, but they can be used to select the correct alignments. Alignments with a probability score <0.015 were selected as the predicted alignments. The performance figures for each family can be seen in the extreme value distribution part of Table 4. The performance for the THI family is again much worse than for the other families, but the tRNA sensitivity has improved significantly.

Table 5 shows how many good alignments would be missed or found if a fixed number of alignments were considered correct in each scan. With the exception of the THI and tRNA family all the correct alignments are found for all families even if only the best hit is chosen. Since tRNA genes are often clustered in the genome, the number of hits chosen for this family should be higher than for the other families. An example of multiple correct alignments can be seen in Figure 3.

The detection of tRNAs are made harder by the fact that they are clustered. The clustering raises the number of tRNA gene pairs in two sequences being compared. The methods using the extreme value distribution and the Z-value are better able to handle this than simply taking the n -best alignments. The THI family is hard to locate

Table 5. The columns indicate how many good alignments would be missed or found if the number of alignments indicated in the column header were chosen

RNA	Number of hits chosen							
	100	20	10	5	4	3	2	1
Missed								
5S rRNA	0	0	0	0	0	0	0	0
Purine	0	0	0	0	0	0	0	0
THI	0	0	3	7	7	8	8	11
U1	0	0	0	0	0	0	0	0
tRNA	0	6	29	64	82	108	149	190
Found								
5S rRNA	2	2	2	2	2	2	2	2
Purine	5	5	5	5	5	5	5	5
THI	21	21	18	14	14	13	13	10
U1	6	6	6	6	6	6	6	6
tRNA	243	237	214	179	161	135	94	53

for any of the three selection methods. This is most likely due to the less well conserved structure of these elements.

The program was used to predict the localization of the SRP genes. In all the eight scans the correct SRP-alignment was the best scoring alignment. Since the sequences are long (1000 nt) and λ large (~ 300 nt) it was not feasible to do random alignments. When the extreme value distribution was estimated directly from the alignment scores of the unshuffled sequences, seven of the correct alignments had a probability of <0.001 for being random; for the last alignment this probability was 0.028. The probability for the least likely false positive was 0.142.

DISCUSSION

We addressed the problem of finding common local structural motifs in two RNA sequences with low sequence similarity ($<40\%$) as well as having the sequences energetically indistinguishable from their surrounding sequence context. We introduced a Sankoff (1985) based approach to conduct simultaneous mutual scan of two sequences for non-coding RNAs (ncRNAs). This implementation is an extension of the previous FOLDALIGN implementation as it now allows for bifurcated structure, includes RIBOSUM (Klein and Eddy, 2003) like substitution matrices and includes structural energy parameters in score computation. The approach presented here conducts pairwise structural alignments with the same quality as an existing method

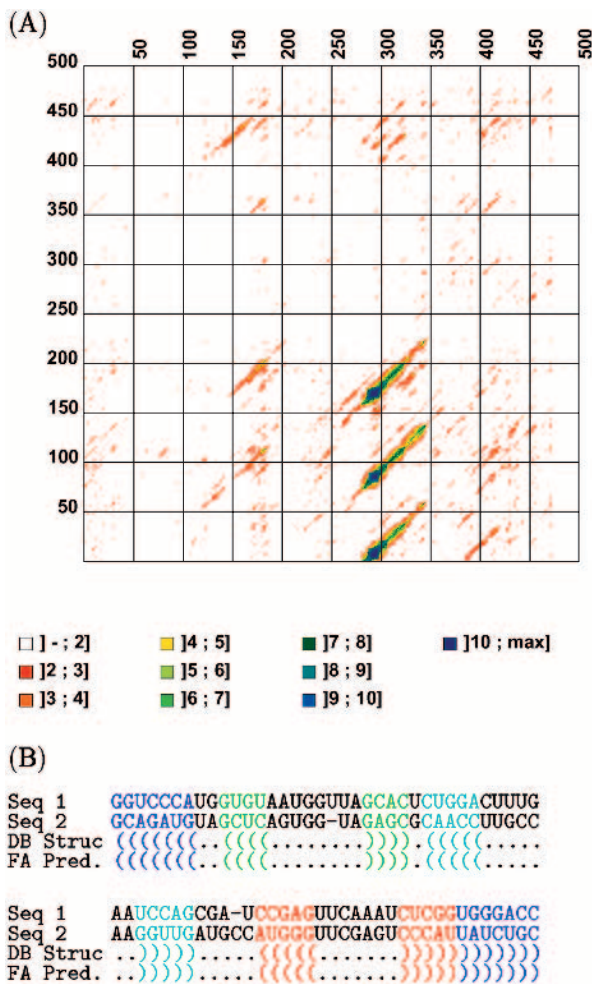


Fig. 3. (A) Shows the Z-value for the best local alignment starting at each pair of positions along the two sequences. The parameters of the Z-value distribution is calculated using the scores of the highest scoring alignment starting at all pairs of positions between the two sequences. The sequence on the x-axis contains one tRNA starting at position 297. The sequence on the y-axis has three tRNAs annotated. These are located at positions 12, 89 and 173. The three regions where the genes align are easily spotted in the plot. The tRNA at position 89 is the one with similarity <40% to the tRNA in the x-axis sequence. The database structures and the FOLDALIGN structure prediction for this region can be seen in (B). The two gaps are placed differently and the final nucleotide is missing but the rest of the predicted structure is correct.

(Mathews and Turner, 2002) and obtains an overall performance of 0.72 using Matthews correlation coefficient.

Furthermore, as the method still maximizes a score it can perform local alignments. The local alignment approach was implemented such that it was memory efficient while conducting mutual scan of two sequences. The ability to make local alignments is desirable as a local alignment approach is expected to be more sensitive in detecting motifs smaller than the window size used in the scanning procedure, than in a global alignment approach (Mathews and Turner, 2002; Hofacker *et al.*, 2004a) if used for scanning.

Different approaches for selecting significant hits were tested, and we found that the probability scores based on the estimated extreme

value distribution worked best. This approach is likely to be refined, so even better hit selection can be made. The obtained performance was 0.82 in sensitivity and 0.88 in positive predictive value.

One limitation is that FOLDALIGN does not deal with structural inserts, which is the likely explanation why it did not perform well on the THI element.

The algorithm can clearly be improved with respect to structural inserts using the principles presented by Eddy (2002b). Also, the ability to select hit region candidates for ncRNAs can be improved.

The approach presented here should also be well suited to be included in a scheme that searches for a common motif in a set of related sequences and should thereby improve the earlier FOLDALIGN implementation for multiple sequences (Gorodkin *et al.*, 1997b, 2001c). In fact, the greedy algorithm might be completely replaced by other methods, such as the stochastic context-free grammar implementation Infernal (Eddy, 2002b).

In conclusion, the new version of FOLDALIGN applies well to many classes of low-sequence similarity ncRNA sequences and complements existing methods for computational searches for ncRNAs.

ACKNOWLEDGEMENTS

This work was supported by the Danish Technical Research Council, the Ministry of Food, Agriculture and Fisheries, and the Danish Center for Scientific Computing. We thank Dave Mathews for the help with Dynalign.

REFERENCES

- Altschul, S. *et al.* (2001) The estimation of statistical parameters for local alignment score distributions. *Nucleic Acids Res.*, **29**, 351–361.
- Benson, D. *et al.* (2004) GenBank: update. *Nucleic Acids Res.*, **32**, 23–26.
- Brown, M. (2000) Small subunit ribosomal RNA modeling using stochastic context-free grammars. In Altman, R., Bailey, T.L., Bourne, P., Gribskov, M., Lengauer, T., Shindyalov, I.N., Eyck, L. F.T. and Weissig, H. (eds), *Proceedings of the Eighth International Conference on Intelligent Systems in Molecular Biology*. AAAI/MIT Press, Menlo Park, CA, pp. 57–66.
- Eddy, S.R. (2002a) Computational genomics of noncoding RNA genes. *Cell*, **109**, 137–140.
- Eddy, S.R. (2002b) A memory-efficient dynamic programming algorithm for optimal alignment of a sequence to an RNA secondary structure. *BMC Bioinformatics*, **3**, 18.
- Gorodkin, J. *et al.* (1997a) Finding common sequence and structure motifs in a set of RNA sequences. In Gaasterland, T., Karp, P., Karplus, K., Ouzounis, C., Sander, C. and Valencia, A. (eds), *Proceedings of the Fifth International Conference on Intelligent Systems in Molecular Biology*. AAAI/MIT Press, Menlo Park, CA, pp. 120–123.
- Gorodkin, J. *et al.* (1997b) Finding the most significant common sequence and structure motifs in a set of RNA sequences. *Nucleic Acids Res.*, **25**, 3724–3732.
- Gorodkin, J. *et al.* (2001a) Semi-automated update and cleanup of structural RNA databases. *Bioinformatics*, **17**, 642–645.
- Gorodkin, J. *et al.* (2001b) A mini-greedy algorithm for faster structural RNA stem-loop search. *Genome Informatics*, **12**, 184–193.
- Gorodkin, J. *et al.* (2001c) Discovering common stem-loop motifs in unaligned RNA sequences. *Nucleic Acids Res.*, **29**, 2135–2144.
- Griffiths-Jones, S. *et al.* (2003) Rfam: an RNA family database. *Nucleic Acids Res.*, **31**, 439–441.
- Henikoff, S. and Henikoff, J.G. (1992) Amino acid substitution matrices from protein blocks. *Proc. Natl Acad. Sci. USA*, **89**, 10915–10919.
- Heyer, L.J. (2000) A generalized erdős-rényi law for sequence analysis problems. *Methodol. Comput. Appl. Probability*, **2**, 309–329.
- Hofacker, I.L. *et al.* (2002) Secondary structure prediction for aligned RNA sequences. *J. Mol. Biol.*, **319**, 1059–1066.
- Hofacker, I. *et al.* (2004a) Alignment of RNA base pairing probability matrices. *Bioinformatics*, **20**, 2222–2227.
- Hofacker, I.L. *et al.* (2004b) Prediction of locally stable RNA secondary structures for genome-wide surveys. *Bioinformatics*, **20**, 186–190.
- Holmes, I. (2004) A probabilistic model for the evolution of RNA structure. *BMC Bioinformatics*, **5**, 166.

- Huttenhofer, A. et al. (2002) RNomics: identification and function of small, non-messenger RNAs. *Curr. Opin. Chem. Biol.*, **6**, 835–843.
- Ji, Y. et al. (2004) A graph theoretical approach for predicting common RNA secondary structure motifs including pseudoknots in unaligned sequences. *Bioinformatics*, **20**, 1591–1602.
- Karlin, S. and Altschul, S. (1990) Methods for assessing the statistical significance of molecular sequence features by using general scoring schemes. *Proc. Natl Acad. Sci. USA*, **87**, 2264–2268.
- Klein, R.J. and Eddy, S.R. (2003) RSEARCH: finding homologs of single structured RNA sequences. *BMC Bioinformatics*, **4**, 44.
- Knudsen, B. and Hein, J.J. (1999) A method to combine a set of alignments in one better alignment. *Bioinformatics*, **15**, 122–130.
- Knudsen, B. et al. (2004) Evolutionary rate variation and RNA secondary structure prediction. *Comput. Biol. Chem.*, **28**, 219–226.
- Kubodera, T. et al. (2003) Thiamine-regulated gene expression of *Aspergillus oryzae* thiA requires splicing of the intron containing a riboswitch-like domain in the 5'-UTR. *FEBS Lett.*, **555**, 516–520.
- Lagos Quintana, M. et al. (2001) Identification of novel genes coding for small expressed RNAs. *Science*, **294**, 853–858.
- Lau, N. et al. (2001) An abundant class of tiny RNAs with probable regulatory roles in *Caenorhabditis elegans*. *Science*, **294**, 858–862.
- Lee, R. and Ambros, V. (2001) An extensive class of small RNAs in *Caenorhabditis elegans*. *Science*, **294**, 862–864.
- Lowe, T. and Eddy, S. (1997) tRNAscan-SE: a program for improved detection of transfer RNA genes in genomic sequence. *Nucleic Acids Res.*, **25**, 955–964.
- Lowe, T. and Eddy, S. (1999) A computational screen for methylation guide snoRNAs in yeast. *Science*, **283**, 1168–1171.
- Macke, T. et al. (2001) RNAMotif, an RNA secondary structure definition and search algorithm. *Nucleic Acids Res.*, **29**, 4724–4735.
- Mandal, M. and Breaker, R. (2004) Adenine riboswitches and gene activation by disruption of a transcription terminator. *Nat. Struct. Mol. Biol.*, **11**, 29–35.
- Mandal, M. et al. (2003) Riboswitches control fundamental biochemical pathways in *Bacillus subtilis* and other bacteria. *Cell*, **113**, 577–586.
- Matthews, B.W. (1975) Comparison of the predicted and observed secondary structure of T4 phage lysozyme. *Biochem. Biophys. Acta*, **405**, 442–451.
- Mathews, D. and Turner, D. (2002) Dynalign: an algorithm for finding the secondary structure common to two RNA sequences. *J. Mol. Biol.*, **317**, 191–203.
- Mathews, D. et al. (1999) Expanded sequence dependence of thermodynamic parameters improves prediction of RNA secondary structure. *J. Mol. Biol.*, **288**, 911–940.
- Miranda Rios, J. et al. (2001) A conserved RNA structure (thi box) is involved in regulation of thiamin biosynthetic gene expression in bacteria. *Proc. Natl Acad. Sci. USA*, **98**, 9736–9741.
- Olsen, R. et al. (1999) Rapid assessment of extremal statistics for gapped local alignment. *Proc. Int. Conf. Intell Syst. Mol. Biol.*, **1999**, 211–222.
- Pavesi, G. et al. (2004) RNAprofile: an algorithm for finding conserved secondary structure motifs in unaligned RNA sequences. *Nucleic Acids Res.*, **32**, 3258–3269.
- Perriquet, O. et al. (2003) Finding the common structure shared by two homologous RNAs. *Bioinformatics*, **19**, 108–116.
- Rivas, E. and Eddy, S. (2001) Noncoding RNA gene detection using comparative sequence analysis. *BMC Bioinformatics*, **2**, 8.
- Rodionov, D. et al. (2002) Comparative genomics of thiamin biosynthesis in prokaryotes. New genes and regulatory mechanisms. *J. Biol. Chem.*, **277**, 48949–48959.
- Rosenblad, M.A. et al. (2003) SRPDB (signal recognition particle database). *Nucleic Acids Res.*, **31**, 363–364.
- Sankoff, D. (1985) Simultaneous solution of the RNA folding, alignment and postsequence problems. *SIAM J. Appl. Math.*, **45**, 810–825.
- Sprinzl, M. et al. (1998) Compilation of tRNA sequences and sequences of tRNA genes. *Nucleic Acids Res.*, **26**, 148–153.
- Sudarsan, N. et al. (2003) An mRNA structure in bacteria that controls gene expression by binding lysine. *Genes Dev.*, **17**, 2688–2697.
- Szymanski, M. et al. (2002) 5S Ribosomal RNA Database. *Nucleic Acids Res.*, **30**, 176–178.
- Van de Peer, Y. et al. (1994) Database on the structure of small ribosomal subunit RNA. *Nucleic Acids Res.*, **22**, 3488–3494.
- Washietl, S. and Hofacker, I.L. (2004) Consensus folding of aligned sequences as a new measure for the detection of functional RNAs by comparative genomics. *J. Mol. Biol.*, **342**, 19–30.
- Weinberg, Z. and Ruzzo, W.L. (2004a) Exploiting conserved structure for faster annotation of non-coding RNAs without loss of accuracy. *Bioinformatics*, **20**(Suppl 1), I334–I341.
- Weinberg, Z. and Ruzzo, W.L. (2004b) Faster genome annotation of non-coding RNA families without loss of accuracy. In *Proceedings of the Eighth Annual International Conference on Computational Molecular Biology (RECOMB)*, ACM Press, pp. 243–251.
- Winkler, W. et al. (2002) An mRNA structure that controls gene expression by binding FMN. *Proc. Natl Acad. Sci. USA*, **99**, 15908–15913.
- Workman, C. and Krogh, A. (1999) No evidence that mRNAs have lower folding free energies than random sequences with the same dinucleotide distribution. *Nucleic Acids Res.*, **27**, 4816–4822.
- Xia, T. et al. (1998) Thermodynamic parameters for an expanded nearest-neighbor model for formation of RNA duplexes with Watson–Crick base pairs. *Biochemistry*, **37**, 14719–14735.
- Zuker, M. and Stiegler, P. (1981) Optimal computer folding of large RNA sequences using thermodynamic and auxiliary information. *Nucleic Acids Res.*, **9**, 133–148.
- Zuker, M., Mathews, D.H. and Turner, D.H. (1999) Algorithms and thermodynamics for RNA secondary structure prediction: a practical guide. In Clark, J.B.B. (ed.), *RNA Biochemistry and Biotechnology*. NATO ASI Series, Kluwer Academic Publishers, Dordrecht, NL, pp. 11–43.
- Zwieb, C. (1996) The uRNA database. *Nucleic Acids Res.*, **24**, 76–79.